CAV-Bench: Can Code Agents Complete Long-Horizon C Program Verification?

Anonymous submission

Abstract

As code agents move from single-response code generation to long-horizon, repository-level workflows, their evaluation must evolve accordingly. Yet most benchmarks judge correctness via incomplete input-output tests. Formal C verification offers strong assurance and demands iterative, feedback-driven work across tightly coupled stages where failures propagate upstream. This makes it well suited to benchmarking code agents. However, existing benchmarks usually evaluate only individual stages, and few target C programs. We introduce CAV-Bench, to our knowledge the first benchmark of formally verified HumanEval implementations in C. It contains 140 human-reviewed tasks that connect executable C programs with contracts, auxiliary annotations, and Rocq proofs. CAV-Bench defines three agent tasks: a primary task with a fixed contract, requiring completion of annotations and proof; an end-to-end task that additionally covers contract generation; and a proof-only task that isolates the final proof stage. All submissions are checked by a deterministic audit. Across frontier agent-model configurations and reasoning-effort levels, the strongest system verifies only 57.1% of the 140 primary-task instances within 90 minutes; other tasks exhibit similar trends. Successful runs require millions of input tokens, repeated repository interaction, and iterative refinement, showing that reliable and efficient formal verification remains a major challenge for current code agents.

Introduction

As code agents rapidly advance, evaluation evolves along two dimensions: from evaluating models that generate code in a single response to agents that sustain tool-driven interaction, and from function-level generation to repository-level workflows. HumanEval and MBPP represent the former setting, whereas SWE-bench evaluates repository-scale bug repair and Terminal-Bench covers command-line agent tasks (Chen et al. 2021; Austin et al. 2021; Jimenez et al. 2024; Merrill et al. 2026).

At a high level, program verification builds a machine-checkable chain from what a program should do to what its implementation actually does. This chain comprises three stages. *Stage I: Contract* states the program's intended behavior through pre-/postconditions. *Stage II: Auxiliary Annotation* adds facts needed at intermediate program points, such as loop invariants. A verification-condition generator (VCG) then automatically analyzes the implementation, contract, and auxiliary annotations to produce proof goals. *Stage III: Proof* constructs machine-checkable proofs for these goals, in our case, Rocq scripts. Once Rocq accepts all proofs, the implementation is formally shown to satisfy its contract. These stages are tightly coupled. A failed proof may require repairing the proof or revising the annotations, after which VCG is rerun and the new proof goals are checked again. While the contract may also be revised in the general case, our primary task fixes it and focuses on the annotation–proof interplay. We additionally provide an end-to-end task that requires generating the contract as well.

This iterative, feedback-driven process, where verification benefits greatly from holistic awareness of the surrounding verification context (e.g., inductive predicates and reusable proof lemmas), makes formal C verification particularly well suited to repository-level agent evaluation. It combines program-semantic reasoning and theorem proving, exercising both code and mathematical capabilities. A deterministic audit checks whether the agent succeeds and prevents cheating.

Existing formal-verification benchmarks, however, usually evaluate only part of this process. Some focus on generating contracts or auxiliary annotations and rely on automated verifiers to complete the proof (Le-Cong, Le, and Murray 2025; Loughridge et al. 2025; Sun et al. 2024). Other benchmarks focus on proof generation. Whether the implementation and contract are provided or generated by the model, they are fixed before proof begins, and the model is evaluated only on constructing the proof (Thakur et al. 2025; Ye et al. 2026; Miranda et al. 2026; Dougherty and Mehta 2025; Xu et al. 2026; Xin et al. 2026). Neither paradigm evaluates whether a code agent can iteratively coordinate auxiliary annotations and proofs, including revising the auxiliary annotations when a failed proof reveals a missing fact.

C program verification is particularly challenging because the language provides no native contract or proof layer and exposes low-level execution details. Contracts and auxiliary annotations must account for not only the intended functional behavior but also low-level properties, such as pointer aliasing, mutable arrays and strings, memory ownership, integer over-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

flow, and other forms of undefined behavior. The proof must connect these low-level facts back to the top-level contract. When a proof fails, the agent may need to repair the proof or strengthen the annotations, making even short functions a long-horizon task.

We introduce CAV-BENCH, to our knowledge the first benchmark of formally verified HumanEval implementations in C. It contains 140 human-reviewed and formally verified C programs with complete contract, annotations, and proof artifacts as ground-truth references. In its primary task, CAV-BENCH gives an agent a repository containing the C implementation and contract to be verified, and asks the agent to complete the auxiliary annotation and proof stages within the repository. Unlike benchmarks that ask a model to synthesize a new program, CAV-BENCH evaluates whether an agent can formally verify a given C implementation against its contract.

Figure 1 summarizes the workflow. The primary task fixes the contract and evaluates the auxiliary annotation and proof stages. We additionally provide an end-to-end task that evaluates all three stages and a proof-only task that supplies the contract and auxiliary annotations to isolate the proof stage. Across all tasks, a deterministic audit checks the submitted artifacts and decides success rather than relying on the agent's own claim.

We conduct extensive experiments across frontier code-agent and model configurations and multiple reasoning-effort levels. Even the strongest evaluated configuration, Codex with GPT-5.5 at xhigh effort, verifies only 57.1% of the 140 primary-task instances within the 90-minute budget. Accepted runs still require tens of minutes, millions of input tokens, and repeated repository and checker interaction. These results leave substantial room to improve not only verification accuracy, but also trajectory efficiency, stopping behavior, and time and token cost.

Our contributions are:

- **A HumanEval benchmark for formal C verification:** 140 tasks pairing natural-language problem descriptions with executable C implementations (adapted from C++ as no C version existed) and their formal contracts. Verification revealed discrepancies in several original solutions.
- **Human-in-the-loop construction of fully verified ground truth:** experts guide a code agent to produce complete auxiliary annotations and Rocq proofs, all checked by a deterministic audit.
- **Three agent tasks with experimental baselines:** contract-to-proof, end-to-end, and proof-only tasks, plus large-scale measurements of yield, reasoning effort, stage coupling, termination failures, and tool-use iteration.

Preliminaries

Program verification. Program verification formally checks whether an implementation satisfies its contract. Contract gives pre- and post-conditions specifying the intended behavior. Auxiliary annotations, notably loop invariants, supply intermediate facts bridging the precondition to the post-condition. A verification condition generator (VCG) then performs program reasoning on the implementation, contract, and annotations to produce proof obligations, purely logical formulas independent of control flow. Proof discharges these obligations using a proof assistant. If all pass, verification succeeds; if not, it may require repairing the proof or strengthening the annotation, after which the VCG is rerun.

QCP. QCP (Wu et al. 2025) is the verification environment we use throughout this work. It is a separation logic-based verifier for C, able to describe memory layouts and changes. Given an implementation, contract, and annotations, QCP invokes its VCG to generate proof obligations in separation logic, which are then discharged in Rocq, where VCG and proof checking are integrated. This unified loop provides deterministic feedback, VCG errors or proof failures supporting iterative revision across stages.

Example. HumanEval Problem 52, `below_threshold`, returns whether every array element is below `t`. The contract specifies functional correctness (via predicates `pre`, `post`) and memory safety via `IntArray::full`. The annotation provides a loop invariant. The VCG generates proof obligations: the invariant must hold on entry, be preserved by each iteration, and imply the postcondition on exit. The proof discharges them in Rocq.

```
int p052_below_threshold(int *l, int l_size, int t)
CONTRACT
/*@ With xs
 Require 0 <= l_size && l_size < INT_MAX && pre(xs) &&
 IntArray::full(l, l_size, xs)
 Ensure ((__return != 0) && post(xs,t,true) ||
 (__return == 0) && post(xs,t,false)) &&
 IntArray::full(l, l_size, xs) */
{
AUXILIARY ANNOTATION
 /*@ Inv Assert 0 <= i && i <= l_size && pre(xs) &&
 (forall (k: Z), (0 <= k && k < i) => Znth(k,xs,0) < t) &&
 IntArray::full(l, l_size, xs) */
  for (i = 0; i < l_size; i++)
    if (l[i] >= t) return 0;
  return 1;
}

PROOF (EXCERPT) Proof. pre_process; ...; lia. Qed.
```

Related Work

Code-generation benchmarks. Function-level benchmarks such as HumanEval, MBPP, APPS, and CodeContests usually evaluate generated programs through tests (Chen et al. 2021; Austin et al. 2021; Hendrycks et al. 2021; Li et al. 2022). MultiPL-E, ClassEval, LiveCodeBench, and BigCodeBench extend coverage across languages, program structures, time, and realistic library use (Cassano et al. 2022; Du et al. 2023; Jain et al. 2025; Zhuo et al. 2025). RepoBench adds repository context, while SWE-agent, SWE-bench, SWE-Bench Pro, and Terminal-Bench evaluate tool-using agents on repository or command-line tasks (Liu, Xu, and McAuley 2024; Yang et al. 2024; Jimenez et al. 2024; Deng et al. 2026; Merrill et al. 2026). CAV-BENCH adopts this repository interaction setting but asks agents to formally verify a fixed C implementation rather than generate or repair code.

Theorem-proving benchmarks. Recent mathematical theorem-proving benchmarks are centered largely on Lean. MiniF2F and FIMO formalize Olympiad-level problems, ProofNet targets undergraduate mathematics, and PutnamBench extends competition mathematics across Lean, Isabelle, and Coq (Zheng, Han, and Polu 2021; Liu et al. 2023; Azerbayev et al. 2023; Tsoukalas et al. 2024). CoqGym and CoqStoq instead evaluate proof synthesis across open-source Coq

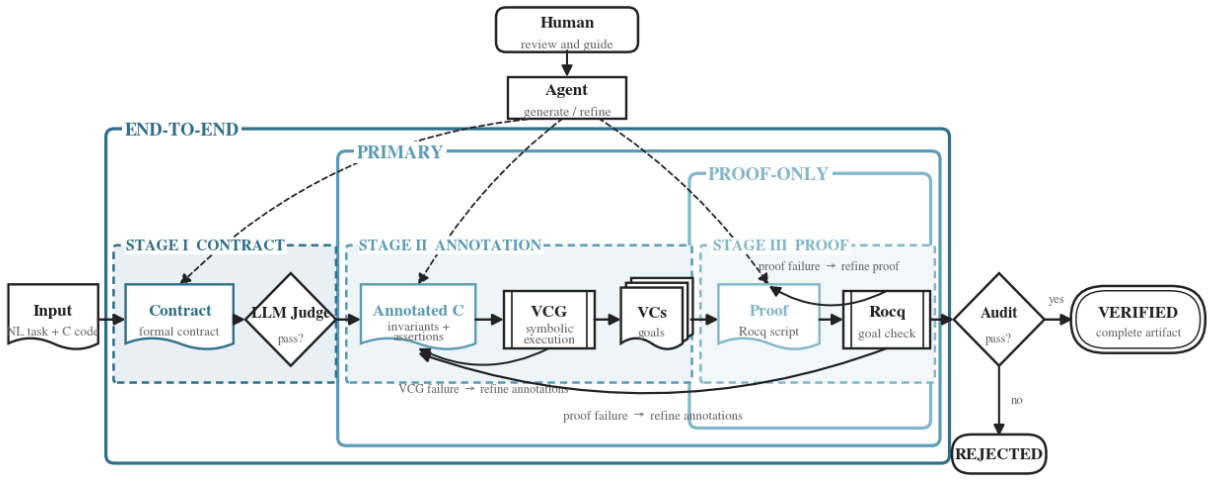


Figure 1: The CAV-Bench construction and evaluation pipeline.

projects, with CoqStoq introduced alongside Rango (Yang and Deng 2019; Thompson et al. 2025). These benchmarks evaluate theorem or proof-state completion rather than a verification workflow rooted in executable programs that couples auxiliary annotations with proofs.

Program-verification benchmarks. Program-verification benchmarks cover complementary parts of the workflow. FormalBench, Coins, and specification-inference work emphasize contracts or semantic properties, while InvBench focuses on auxiliary annotations (Cosler et al. 2023; Wen et al. 2024; Ma et al. 2025; Le-Cong, Le, and Murray 2025; Yang et al. 2026; Wei et al. 2026). DafnyBench, CloverBench, CLEVER, VERINA, VeriBench, VeriEquivBench, Vericoding, AlgoVeri, and VeriContest study broader combinations of verifiable code and proofs, while FVAPPS, NTP4VC, and VeriSoftBench emphasize proofs or proof repositories (Loughridge et al. 2025; Sun et al. 2024; Thakur et al. 2025; Ye et al. 2026; Miranda et al. 2026; Zeng et al. 2026; Bursuc et al. 2025; Zhao et al. 2026; Xie et al. 2026; Dougherty and Mehta 2025; Xu et al. 2026; Xin et al. 2026). These benchmarks do not evaluate the coupled auxiliary-annotation and proof workflow for C programs, where auxiliary annotations must explicitly characterize memory.

C program verification. Many tools have been developed for C program verification, using different Contract languages and proof workflows. Frama-C uses ACSL to express quantified conditions, program behaviors, and memory changes, and its WP plugin relies mainly on automated provers (Kirchner et al. 2015). VeriFast instead uses separation logic to express memory ownership and recursive data structures, which it checks through symbolic execution (Jacobs et al. 2011). QCP combines memory-aware Contracts and Rocq definitions with an explicit Rocq Proof, exposing the interaction between Auxiliary Annotation and Proof that CAV-Bench evaluates (Wu et al. 2025).

Table 1 summarizes this comparison. Unlike the compared benchmarks, CAV-Bench requires a repository-running agent to complete the coupled auxiliary annotation and proof workflow for a fixed C implementation and contract.

CAV-Bench

CAV-Bench turns HumanEval programs and their contracts into 140 repository-level C verification tasks. We construct the benchmark by collecting and adapting the source materials, building verified ground truth, and defining three task settings.

Benchmark Data Collection

Each CAV-Bench task is built from a natural language description and a C implementation, together with a complete ground-truth artifact consisting of a QCP contract, auxiliary annotations, and Rocq proofs. The descriptions come from HumanEval (Chen et al. 2021), and the C implementations are adapted from the corresponding C++ implementations in HumanEval-X (Zheng et al. 2023), with modifications to accommodate C's data representation, library operations, and memory management. During verification of the adapted C implementations, we found several cases whose behavior deviated from the original HumanEval descriptions. We corrected the implementations to match the descriptions and report the affected tasks in the supplementary material. For contracts, we adapt the human-written Rocq contracts released with Coins (Yang et al. 2026) (which specify functional correctness only) into QCP contracts, adding conditions to establish memory safety and the absence of runtime errors (RTEs) in C. We also retain or adapt any mathematical helper definitions required by the contracts.

Of the 164 HumanEval problems, 140 enter CAV-Bench. We mainly exclude floating-point programs because their numerical reasoning is beyond the practical scope of the current QCP workflow.

Agents work in the QCP repository, where they can run the VCG and Rocq, consult documentation and examples, and import the provided Rocq libraries. The QCP contract with required Rocq definitions, and auxiliary annotations are

Table 1: Comparison with program-verification benchmarks. Filled marks indicate components generated and evaluated in the benchmark; "GT Verified" indicates that the complete ground-truth artifact passes the corresponding checker.

Benchmark	Language / verifier	Spec.	Inv.	Other Ann.	ITP proof	GT Verified
FormalBench	Java / OpenJML	●	●	●	○	○
InvBench	C / UAutomizer	○	●	○	○	○
DafnyBench	Dafny	○	○	●	○	●
CloverBench	Dafny	●	●	●	○	●
CLEVER	Lean	●	○	○	●	○
VERINA	Lean	●	○	○	●	○
VeriBench	Lean	●	○	○	●	○
AlgoVeri	Dafny / Verus / Lean	○	●	●	●	○
FVAPPS	Lean	○	○	○	●	○
NTP4VC	Lean / Rocq / Isabelle	○	○	○	●	●
VeriSoftBench	Lean	○	○	○	●	●
CAV-Bench	C / Rocq	●	●	●	●	●

Table 2: Statistics of the verified CAV-Bench corpus.

Statistic	Total	Mean / task
HumanEval source problems	164	–
Verified benchmark problems	140	–
C code LOC	4,076	29.1
Function-contract LOC	3,114	22.2
Auxiliary-annotation LOC	10,270	73.4
Task-specific Rocq definition LOC	8,083	57.7
Manual Rocq proof LOC	62,812	448.7
Generated verification conditions	5,562	39.7

exposed only when required by the task setting. Ground-truth Rocq proofs are never exposed to agents.

Ground-Truth Construction

We construct a complete verification result for every included problem through the human-agent workflow in Figure 1. Starting from a C implementation and the contract derived from Coins, human experts guide a code agent to produce auxiliary annotations and proof. QCP's VCG analyzes the annotated C program and generates proof goals, which Rocq then checks (Wu et al. 2025). The agent iteratively revises the auxiliary annotations and proof based on feedback from these tools.

Human experts review intermediate results, provide semantic guidance when the agent stalls, and check the final contract against the HumanEval description and the adapted Coins contract. They prevent the agent from using commands that bypass proof obligations (e.g., `admit` or `axiom`), but do not write task-specific proofs or annotations themselves.

A task is included only when rerunning VCG regenerates all proof goals, Rocq accepts every proof, and the experts confirm that the contract preserves the intended behavior. Because this construction process includes human effort that is not fully recorded, its time and token costs are not directly comparable with the fully automated evaluation runs.

Task Design

The 140 verified results define the three task settings shown in Figure 1. Our primary *contract-to-proof* task provides a fixed, human-reviewed, formally verified ground-truth contract and asks the agent to complete the auxiliary annotation and proof stages. Fixing the contract gives every agent the same intended behavior while focusing on the interaction between auxiliary annotations and proofs.

We additionally provide two tasks. The *end-to-end* task asks the agent to complete all three stages. Because different contracts may correctly describe the same behavior, contract quality cannot be determined by a single automatic check. We therefore use a lightweight rubric to evaluate whether the generated contract adequately covers the intended behavior and safety requirements, with details provided in the supplementary material. The *proof-only* task supplies the contract and the ground-truth auxiliary annotations, allowing us to evaluate proof construction separately.

Evaluated agents receive no expert guidance or task-specific proof hints. The deterministic audit reruns VCG and Rocq, checks that the fixed contract and executable implementation remain unchanged, detects ways of avoiding the required proof, and verifies that the agent did not read disallowed material. Further details are provided in the supplementary material.

Experiments and Results

Experimental setup. We evaluate code agents on the primary contract-to-proof task across all 140 tasks. Each agent receives a fixed C implementation and contract and works in the QCP repository to complete the auxiliary annotations and proof. All agents receive the same general workflow guidance and may use VCG, Rocq, the provided documentation and examples, and the Rocq libraries, but receive no task-specific hints. Each run has a 90-minute wall-clock budget. A deterministic audit decides success by rerunning VCG and Rocq and checking the submitted artifact as described.

For each model and reasoning-effort configuration, we report audited successes (W), non-timeout failures (F), and timeouts (T), conservatively counting missing runs as timeouts. An audited completion counts as a success only if its

Table 3: Primary-task results over 140 tasks. Missing runs are counted as timeouts. Each cost cell reports the audited-success mean followed by the gray non-timeout mean, which includes failed submissions. Operation counts include only recorded trace events, and rounds count assistant messages.

	Outcome			Cost		Trace activity			
Model / effort	$W/140$	F/T	SR (%)	Time (min)	In (M) / Out (k)	Read / Write	Tools	Rocq / VCG	Rounds
Codex									
GPT-5.5 (xhigh)	80/140	31/29	**57.1**	42.8 45.1	9.1/35 9.9/38	79.9/7.1 81.6/9.0	119.7 125.3	12.6/3.0 14.5/3.6	55.8 59.7
GPT-5.5 (medium)	59/140	61/20	42.1	33.4 36.6	7.0/23 9.8/30	65.5/8.1 71.0/14.6	99.0 116.5	11.0/3.3 18.4/4.0	49.3 63.8
GPT-5.4 (xhigh)	53/140	48/39	37.9	38.5 45.8	22.4/78 24.6/85	94.3/25.5 109.2/33.7	171.0 200.5	43.1/3.0 48.8/4.8	49.5 58.8
GPT-5.4 (high)	40/140	96/4	28.6	43.1 39.5	20.3/59 21.8/67	95.8/35.8 101.1/36.2	156.3 169.4	31.4/2.2 32.9/4.9	52.1 47.7
GPT-5.4 (medium)	20/140	120/0	14.3	21.2 23.9	15.1/41 15.2/47	66.0/30.6 77.4/29.0	125.1 132.9	33.8/2.5 27.8/5.4	31.9 36.2
GPT-5.4 (low)	16/140	124/0	11.4	21.7 16.2	14.7/36 10.3/31	59.3/29.4 57.3/20.6	125.8 103.6	34.6/2.8 18.9/5.0	29.7 26.6
GPT-5.4-mini (medium)	8/140	130/2	5.7	36.4 38.4	18.7/103 19.2/111	89.4/36.0 102.4/33.6	171.2 181.7	45.6/2.0 35.2/9.5	81.1 95.6
Claude Code									
GLM-5.2 (thinking)	30/140	77/33	21.4	28.3 48.6	11.5/93 15.4/105	79.5/27.1 119.8/39.2	105.8 151.9	31.0/3.3 44.2/6.5	89.8 136.3
DeepSeek-V4-Pro (thinking)	5/140	55/80	3.6	33.2 69.8	14.5/73 27.4/69	81.0/50.8 100.4/54.2	142.6 175.2	46.4/5.0 43.6/14.5	130.0 149.2
Kimi Code									
kimi-2.7-code (thinking)	18/140	99/23	12.9	38.8 64.8	17.0/86 30.3/147	114.1/42.5 185.8/88.5	166.3 274.6	44.5/3.6 86.6/9.9	157.6 264.8

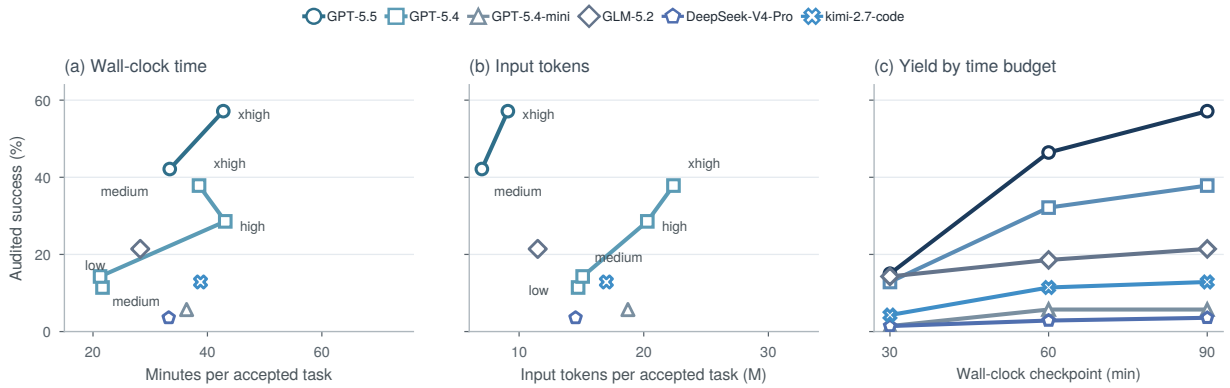


Figure 2: Success and successful-run cost across models and reasoning-effort levels. Color and marker shape jointly identify each model. Panels (a) and (b) plot audited success against mean wall-clock time and input tokens per accepted task; connected points vary reasoning effort within a GPT model family. Panel (c) reports cumulative audited success at 30, 60, and 90 minutes using the strongest GPT effort level and the fixed configuration of each other model.

recorded end-to-end wall-clock time is at most 5,400 seconds; audited completions exceeding this limit are counted as timeouts. Every success rate uses all 140 tasks as its denominator. In each cost cell of Table 3, the first value averages over successful runs and the gray value averages over all non-timeout runs, including failed attempts. We use a model's official harness when available and Claude Code otherwise; RQ5 studies the effect of harness choice.

RQ1: How Well Do Code Agents Complete C Verification?

Table 3 shows that the strongest configuration, GPT-5.5 at xhigh effort, verifies only 80 of the 140 tasks (57.1%) within 90 minutes. Even these short HumanEval-derived functions require repeated repository exploration, auxiliary-annotation revisions, proof repairs, and VCG and Rocq checks. Figure 2(c) shows that audited success continues to increase from 30 to 60 and 90 minutes for the strongest configurations, confirming that formal verification requires long-horizon interaction. However, additional time alone is insufficient. Many agents stop before timeout without producing an accepted result, showing that they often fail to recognize when verification remains incomplete. We should use deterministic loop engineering to make termination depend on external verification rather than model judgment. The interaction counts in Table 3 further confirm that success requires sustained repository work, while the gray values show that failed runs can remain expensive.

Finding 1. Even short C functions require long-horizon verification: the strongest configuration solves only 57.1% of tasks within 90 minutes, and many agents terminate before the timeout without passing the external audit.

RQ2: How Do Capability and Reasoning Effort Affect Performance?

Figure 2 and the trace columns of Table 3 reveal a joint capability–effort scaling pattern. Greater model capability generally reduces the time, token demand, tool use, and exploratory work needed to reach an accepted solution, whereas increasing reasoning effort within a model sustains longer trajectories and raises local inference cost. Overall, higher reasoning effort tends to increase resource use, but fewer unproductive steps partially offset the added inference cost. Reasoning effort also controls persistence. Low-effort configurations disproportionately terminate before the wall-clock limit, while higher effort keeps the agent working through checker failures and repository revisions. Thus, effort changes not only the quality of individual decisions but also whether the agent continues an unresolved task long enough to solve it.

Finding 2. Greater capability makes agent trajectories more efficient, whereas greater reasoning effort increases persistence and resource use. However, the marginal resource cost of additional reasoning declines as stronger reasoning reduces unproductive exploration.

RQ3: Where Does Difficulty Arise Across Verification Stages?

Figure 3 compares the three task settings using GPT-5.4 at medium effort. The proof stage takes most of the successful-run time, but much of its difficulty comes from the interaction between auxiliary annotations and proofs. Missing or incorrect auxiliary annotations can produce proof goals that cannot be proved, while proof failures often reveal which facts must be added to the auxiliary annotations. Supplying verified auxiliary annotations in the proof-only task improves success from 20 to 30 tasks. Among the 12 tasks accepted in both settings, it also reduces time, input tokens, output tokens, and tool calls.

The contract stage requires less runtime but is difficult to evaluate. Different contracts may correctly describe the same behavior, and an automatically accepted contract may still be too weak or fail to capture the intended behavior. The agent therefore lacks the deterministic feedback available in the auxiliary annotation and proof stages, and many generated contracts are rejected before reaching the later stages.

Finding 3. Proof takes most of the runtime, but the interaction between auxiliary annotations and proofs drives much of the difficulty; contract generation remains hard because it lacks deterministic feedback.

RQ4: Why Do Agent Runs Fail?

Table 4 reports failure categories that may overlap. We classify a run as a VCG-stage failure when VCG produces no proof goals. Otherwise, we classify it as reaching the proof stage. Audit failures are counted separately.

Table 4: Primary-task failures by model configuration. Categories may overlap. F/T denotes non-timeout failures/timeouts, with missing runs counted as timeouts; $P/I/R$ denotes invalid proofs, fixed-input modifications, and reads of disallowed material.

Model / effort	Outcome F/T	Stage Proof/VCG	Audit $P/I/R$
Codex			
GPT-5.5 (xhigh)	31/29	29/2	7/13/5
GPT-5.5 (medium)	61/20	57/4	11/19/7
GPT-5.4 (xhigh)	48/39	44/4	6/29/16
GPT-5.4 (high)	96/4	91/5	6/12/28
GPT-5.4 (medium)	120/0	110/10	9/14/28
GPT-5.4 (low)	124/0	102/22	3/18/33
GPT-5.4-mini (medium)	130/2	96/34	0/33/86
Claude Code			
GLM-5.2 (thinking)	77/33	73/4	1/9/55
DeepSeek-V4-Pro (thinking)	55/80	46/9	0/17/39
Kimi Code			
kimi-2.7-code (thinking)	99/23	96/3	5/21/92
Total	**841/230**	**744/97**	**48/185/389**

Of the 841 non-timeout failures, 744 (88.5%) reach the proof stage, while 97 (11.5%) fail at VCG. GPT-5.4-mini and GPT-5.4 at low effort have the most VCG failures, while Kimi-2.7-code and GPT-5.4-mini most frequently read disallowed material. The predominance of proof-stage failures indicates that most difficulties arise after proof obligations are generated. However, because annotations shape these obligations, the category may capture failures in both annotation construction and proof writing.

Audit failures are separate from verification-stage failures. They include bypassing the required proof, modifying the fixed contract or implementation, and reading disallowed material. Their frequency shows that instructions and model-reported completion are insufficient; success must be determined by a deterministic audit.

Finding 4. Most failures reach the proof stage, highlighting the interaction between auxiliary annotations and proofs; frequent audit violations make deterministic auditing necessary.

RQ5: How Does the Agent Harness Affect Performance?

A harness gives the model repository access, runs tool commands, and manages context. The main experiments use Claude Code for GLM-5.2. We take all 30 tasks that GLM-5.2 completes under Claude Code and rerun the same model on them through OpenCode and Codex CLI. This cohort is fixed before inspecting either alternative harness. The implementation, contract, repository, instructions, 90-minute budget, and deterministic audit remain unchanged. The comparison therefore measures how many Claude Code successes

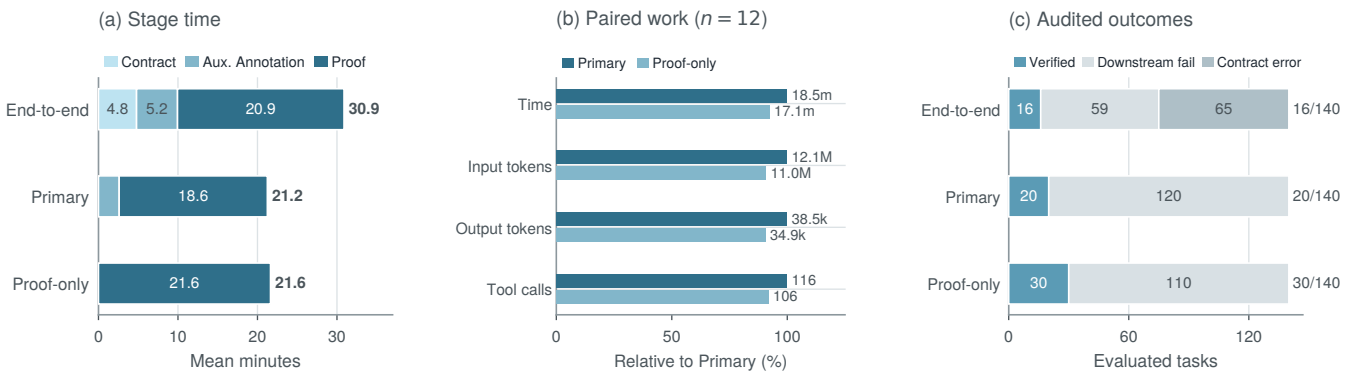


Figure 3: Stage-level results for GPT-5.4 at medium effort. Panel (a) shows the mean time spent in each stage for accepted runs. Panel (b) compares cost on tasks accepted in both the primary and proof-only settings, with each metric normalized to the primary task. Panel (c) reports audited outcomes for the three task settings.

each harness retains, rather than ranking the harnesses over all 140 tasks.

Claude Code accepts 30/30 tasks by construction. Codex CLI retains 23 successes (76.7%), while OpenCode retains 13 (43.3%). Codex CLI has three timeouts and OpenCode has eleven. Changing only the harness can therefore cause the same model to lose previously successful tasks and can change whether an unsuccessful run stops early or uses the full budget.

The harnesses solve different tasks, so their mean cost over all successes is not directly comparable. We instead compare the 12 tasks solved by all three. On this shared set, Claude Code uses the least time, tokens, and tool calls. Codex CLI retains more of the original cohort than OpenCode, but costs more than Claude Code on the shared tasks; OpenCode costs the most. Figure 4 further shows that cross-harness cost can be nearly identical on one task but differ sharply on another. Harness choice is therefore part of the evaluated system, not merely a way to access the model.

Finding 5. Harness choice materially affects both acceptance and cost. Codex CLI performs relatively close to Claude Code, but no alternative harness recovers all Claude Code successes.

Limitations

Program scale. The 140 benchmark tasks are short, single-function HumanEval programs. They do not cover large APIs, concurrent systems, or multi-module verification projects, so the results may not generalize to larger and more complex programs. Nevertheless, formally verifying these programs remains extremely challenging even at this scale.

Model coverage. We evaluate a representative subset of models and effort configurations rather than all available systems. The observed accuracies and rankings should therefore not be extrapolated beyond the evaluated configurations.

Evaluation cost. At public list prices, the evaluated runs represent approximately $6,700 in API-equivalent inference cost. Agentic verification involves long trajectories and repeated model calls, making evaluation expensive even for HumanEval-scale programs. This cost constrains both the scale of the programs and the number of model configurations that we can evaluate.

Conclusion

CAV-Bench provides 140 repository-level tasks for long-horizon C verification under QCP–Rocq. The strongest configuration verifies only 57.1%, with failures across annotation construction, verification-condition generation, and Rocq proofs. Persistent capability and harness effects, together with frequent invalid artifacts, make C verification a rigorous benchmark for feedback-driven code agents and deterministic auditing.

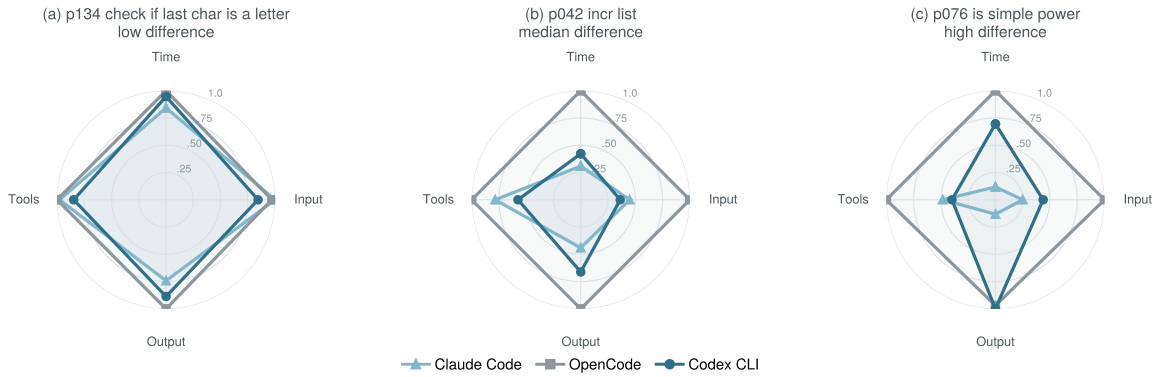


Figure 4: Cost profiles for three GLM-5.2 tasks accepted under all three harnesses, chosen to represent low, median, and high cross-harness differences. Each metric is normalized to the largest value for that task, so smaller values indicate lower cost.

References

Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; and Sutton, C. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732.

Azerbayev, Z.; Piotrowski, B.; Schoelkopf, H.; Ayers, E. W.; Radev, D.; and Avigad, J. 2023. ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. arXiv:2302.12433.

Bursuc, S.; Ehrenborg, T.; Lin, S.; Astefanoaei, L.; Chiosa, I. E.; Kukovec, J.; Singh, A.; Butterley, O.; Bizid, A.; Dougherty, Q.; Zhao, M.; Tan, M.; and Tegmark, M. 2025. A Benchmark for Vericoding: Formally Verified Program Synthesis. arXiv:2509.22908.

Cassano, F.; Gouwar, J.; Nguyen, D.; Nguyen, S.; Phipps-Costin, L.; Pinckney, D.; Yee, M.-H.; Zi, Y.; Anderson, C. J.; Feldman, M. Q.; Guha, A.; Greenberg, M.; and Jangda, A. 2022. MultiPL-E: A Scalable and Extensible Approach to Benchmarking Neural Code Generation. arXiv:2208.08227.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H. P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F. P.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W. H.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A. N.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Cosler, M.; Hahn, C.; Mendoza, D.; Schmitt, F.; and Trippel, C. 2023. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *International Conference on Computer Aided Verification*, 383–396. Springer.

Deng, X.; Da, J.; Pan, E.; He, Y. Y.; Ide, C.; Garg, K.; Lauffer, N.; Park, A.; Rane, C.; Sampath, K.; Krishnan, M.; Kundurthy, S. R.; Hendryx, S. M.; Wang, Z.; Zhang, C. B. C.; Jacobson, N.; Liu, B.; and Kenstler, B. 2026. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? Submitted to ICLR 2026.

Dougherty, Q.; and Mehta, R. 2025. Proving the Coding Interview: A Benchmark for Formally Verified Code Generation. *arXiv preprint arXiv:2502.05714*.

Du, X.; Liu, M.; Wang, K.; Wang, H.; Liu, J.; Chen, Y.; Feng, J.; Sha, C.; Peng, X.; and Lou, Y. 2023. ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation. arXiv:2308.01861.

Hendrycks, D.; Basart, S.; Kadavath, S.; Mazeika, M.; Arora, A.; Guo, E.; Burns, C.; Puranik, S.; He, H.; Song, D.; and Steinhardt, J. 2021. Measuring Coding Challenge Competence With APPS. *arXiv preprint arXiv:2105.09938*.

Jacobs, B.; Smans, J.; Philippaerts, P.; Vogels, F.; Penninckx, W.; and Piessens, F. 2011. VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java. In *NASA Formal Methods*, 41–55. Springer.

Jain, N.; Han, K.; Gu, A.; Li, W.-D.; Yan, F.; Zhang, T.; Wang, S.; Solar-Lezama, A.; Sen, K.; and Stoica, I. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *International Conference on Learning Representations*.

Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. R. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues? In *The Twelfth International Conference on Learning Representations*.

Kirchner, F.; Kosmatov, N.; Prevosto, V.; Signoles, J.; and Yakobowski, B. 2015. Frama-C: A Software Analysis Perspective. *Formal Aspects of Computing*, 27(3): 573–609.

Le-Cong, T.; Le, B.; and Murray, T. 2025. Can LLMs Reason About Program Semantics? A Comprehensive Evaluation of LLMs on Formal Specification Inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 21991–22014. Vienna, Austria: Association for Computational Linguistics.

Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Dal Lago, A.;

Hubert, T.; Choy, P.; de Masson d'Autume, C.; Babuschkin, I.; Chen, X.; Huang, P.-S.; Welbl, J.; Gowal, S.; Cherepanov, A.; Molloy, J.; Mankowitz, D. J.; Robson, E. S.; Kohli, P.; de Freitas, N.; Kavukcuoglu, K.; and Vinyals, O. 2022. Competition-Level Code Generation with AlphaCode. *Science*.

Liu, C.; Shen, J.; Xin, H.; Liu, Z.; Yuan, Y.; Wang, H.; Ju, W.; Zheng, C.; Yin, Y.; Li, L.; Zhang, M.; and Liu, Q. 2023. FIMO: A Challenge Formal Dataset for Automated Theorem Proving. *arXiv preprint arXiv:2309.04295*.

Liu, T.; Xu, C.; and McAuley, J. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *International Conference on Learning Representations*.

Loughridge, C. R.; Sun, Q.; Ahrenbach, S.; Cassano, F.; Sun, C.; Sheng, Y.; Mudide, A.; Misu, M. R. H.; Amin, N.; and Tegmark, M. 2025. DafnyBench: A Benchmark for Formal Software Verification. *Transactions on Machine Learning Research*.

Ma, L.; Liu, S.; Li, Y.; Xie, X.; and Bu, L. 2025. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. In *47th International Conference on Software Engineering (ICSE)*.

Merrill, M. A.; Shaw, A. G.; Carlini, N.; Li, B.; Raj, H.; Bercovich, I.; Shi, L.; Shin, J. Y.; Walshe, T.; Buchanan, E. K.; Shen, J.; Ye, G.; Lin, H.; Poulos, J.; Wang, M.; Nezhurina, M.; Jitsev, J.; Lu, D.; Mastromichalakis, O. M.; Xu, Z.; Chen, Z.; Liu, Y.; Zhang, R.; Chen, L. L.; Kashyap, A.; Uslu, J.-L.; Li, J.; Wu, J.; Yan, M.; Bian, S.; Sharma, V.; Sun, K.; Dillmann, S.; Anand, A.; Lanpouthakoun, A.; Koopah, B.; Hu, C.; Guha, E.; Dreiman, G. H. S.; Zhu, J.; Krauth, K.; Zhong, L.; Muennighoff, N.; Amanfu, R.; Tan, S.; Pimpalgaonkar, S.; Aggarwal, T.; Lin, X.; Lan, X.; Zhao, X.; Liang, Y.; Wang, Y.; Wang, Z.; Zhou, C.; Heineman, D.; Liu, H.; Trivedi, H.; Yang, J.; Lin, J.; Shetty, M.; Yang, M.; Omi, N.; Raoof, N.; Li, S.; Zhuo, T. Y.; Lin, W.; Dai, Y.; Wang, Y.; Chai, W.; Zhou, S.; Wahdany, D.; She, Z.; Hu, J.; Dong, Z.; Zhu, Y.; Cui, S.; Saiyed, A.; Kolbeinsson, A.; Hu, J.; Rytting, C. M.; Marten, R.; Wang, Y.; Dimakis, A.; Konwinski, A.; and Schmidt, L. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arXiv:2601.11868.

Miranda, B.; Daruru, S.; Hersch, E. S.; Zhou, Z.; Nie, A.; Amrollahi, D.; Aniva, L.; Mlauzi, I.; Acharya, K.; Obbad, E.; Soylu, D.; Kirk, W.; Wang, Z. J.; Fronsdal, K.; Li, Y.; Poindexter, J., Donald; Kaushik, R.; Liu, S.; Denisov-Blanch, Y.; Dillmann, S.; Obstbaum, S.; Cuellar, S.; Sarracino, J.; Schaeffer, R.; Tiwari, M.; Lee, D.; Han, B.; and Koyejo, S. 2026. VeriBench: An End-to-End Formal Verification Benchmark for AI Coding Agents in Lean 4. In *3rd AI for Math Workshop at the 43rd International Conference on Machine Learning*.

Sun, C.; Sheng, Y.; Padon, O.; and Barrett, C. W. 2024. Clover: Closed-Loop Verifiable Code Generation. In *AI Verification*, volume 14846 of *Lecture Notes in Computer Science*, 134–155. Springer.

Thakur, A.; Lee, J.; Tsoukalas, G.; Sistla, M.; Zhao, M.; Zetzsche, S.; Durrett, G.; Yue, Y.; and Chaudhuri, S. 2025. CLEVER: A Curated Benchmark for Formally Verified Code Generation. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Thompson, K.; Saavedra, N.; Carrott, P.; Fisher, K.; Sanchez-Stern, A.; Brun, Y.; Ferreira, J. F.; Lerner, S.; and First, E. 2025. Rango: Adaptive Retrieval-Augmented Proving for Automated Software Verification. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 347–359. IEEE Computer Society.

Tsoukalas, G.; Lee, J.; Jennings, J.; Xin, J.; Ding, M.; Jennings, M.; Thakur, A.; and Chaudhuri, S. 2024. PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition. In Globerson, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J.; and Zhang, C., eds., *Advances in Neural Information Processing Systems*, volume 37, 11545–11569. Curran Associates, Inc.

Wei, A.; Suresh, T.; Sun, T.; Wu, H.; Wang, K.; and Aiken, A. 2026. InvBench: Can LLMs Accelerate Program Verification with Invariant Synthesis? In *The Fourteenth International Conference on Learning Representations*.

Wen, C.; Cao, J.; Su, J.; Xu, Z.; Qin, S.; He, M.; Li, H.; Cheung, S.-C.; and Tian, C. 2024. Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification. In *Computer Aided Verification*, volume 14682 of *Lecture Notes in Computer Science*, 302–328. Springer.

Wu, X.; Feng, Y.; Lu, X.; Lin, T.; Liu, K.; Wang, Z.; Wu, S.; Xie, L.; Yang, C.; Zhong, H.; Zhan, N.; Hu, Z.; and Cao, Q. 2025. QCP: A Practical Separation Logic-based C Program Verification Tool. arXiv:2505.12878.

Xie, Z.; Pawagi, M.; Liu, Y.; Rai, A.; Shao, L.; Berberian, J. J.; Che, S.; and Wang, W. 2026. VeriContest: A Competitive-Programming Benchmark for Verifiable Code Generation. arXiv:2605.08553.

Xin, Y.; Chen, Q.; Durrett, G.; and Dillig, I. 2026. VeriSoftBench: Repository-Scale Formal Verification Benchmarks for Lean. arXiv:2602.18307.

Xu, Q.; Luan, X.; Wang, R.; Leang, J. O. J.; Wang, P.; Li, H.; Li, W.; and Watt, C. 2026. Neural Theorem Proving for Verification Conditions: A Real-World Benchmark. In *The Fourteenth International Conference on Learning Representations*.

Yang, F.; Li, X.; Wang, S.; An, J.; Sun, Z.; Feng, S.; Wang, W.; Wang, W.; Zhan, N.; and Xu, F. 2026. How Powerful Are LLMs in Generating Formal Program Specifications? In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K. R.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, volume 37, 50528–50652. Curran Associates, Inc.

Yang, K.; and Deng, J. 2019. Learning to Prove Theorems via Interactive Stepping and Goal-Conditioned Policy. In *International Conference on Machine Learning (ICML)*.

Ye, Z.; Yan, Z.; He, J.; Kasriel, T.; Yang, K.; and Song, D. 2026. VERINA: Benchmarking Verifiable Code Generation. In *The Fourteenth International Conference on Learning Representations*.

Zeng, L.; Che, F.; Huang, X.; Ye, F.; Xu, X.; Yuan, B.; and Fu, J. 2026. VeriEquivBench: An Equivalence Score for Ground-Truth-Free Evaluation of Formally Verifiable Code. In *The Fourteenth International Conference on Learning Representations*.

Zhao, H.; Yang, Z.; Li, J.; He, D.; Li, Z.; Jin, C.; Veeravalli, V. V.; Gupta, A.; and Arora, S. 2026. AlgoVeri: An Aligned Benchmark for Verified Code Generation on Classical Algorithms. In *International Conference on Machine Learning*.

Zheng, K.; Han, J. M.; and Polu, S. 2021. MiniF2F: A Cross-System Benchmark for Formal Olympiad-Level Mathematics. *arXiv preprint arXiv:2109.00110*.

Zheng, Q.; Xia, X.; Zou, X.; Dong, Y.; Wang, S.; Xue, Y.; Wang, Z.; Shen, L.; Wang, A.; Li, Y.; Su, T.; Yang, Z.; and Tang, J. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5673–5684.

Zhuo, T. Y.; Vu, M. C.; Chim, J.; Hu, H.; Yu, W.; Widyasari, R.; Yusuf, I. N. B.; Zhan, H.; He, J.; Paul, I.; Brunner, S.; Gong, C.; Hoang, J.; Zebaze, A.; Hong, X.; Li, W.-D.; Kaddour, J.; Xu, M.; Zhang, Z.; Yadav, P.; Jain, N.; Gu, A.; Cheng, Z.; Liu, J.; Liu, Q.; Wang, Z.; Hui, B.; Muennighoff, N.; Lo, D.; Fried, D.; Du, X.; de Vries, H.; and von Werra, L. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In Yue, Y.; Garg, A.; Peng, N.; Sha, F.; and Yu, R., eds., *International Conference on Learning Representations*, volume 2025, 66602–66656.